

好奇心驅使的能力與志向

我對資訊工程的興趣不是從課堂開始的，而是家裡的自然薰陶。父親雖然從事 **WiFi** 驅動程式開發，但他幾乎不曾刻意教我任何技術。即便如此，在看到 he 使用 **vim** 和 **git** 的日常 **workflow** 後，激起了心中的好奇心。於是，在一次次的練習之後也逐漸習慣了身為工程師該有的快速與效率的工作流。某種程度上，他給我的不是知識，而是一個「身邊有人在工程現場」的環境。國二時開始瀏覽外國論壇 **Reddit**，從中接觸到 **iOS** 越獄與 **Ubuntu**。在 **Ubuntu** 上摸索一年後，我決定挑戰 **Arch Linux**，靠著 **Arch Wiki** 與 **YouTube** 影片從零完成安裝。這些技能沒有人要求我學，純粹是因為想知道 **Linux** 底下到底是怎麼運作的，而這份好奇心一旦啟動就沒有停過。

從使用者到創造者

高一加入電算社，隔年寒假和社員以 **pygame** 合作開發了一款小型遊戲作為社團靜態展的代表作品。這是我第一次在「自己寫好玩」之外的場景裡寫程式。有明確的交付期限、有需要協調的團隊成員、有必須對外展示的完成品。我主導了架構設計與核心開發，但過程中發現技術能力和把事情做完是兩件事：組員對 **git** 不熟導致初期頻繁衝突，我必須在「全部自己來比較快」和「拆成小任務讓大家都能貢獻」之間做出選擇。選擇後者後我才發現犧牲一些時間來為未來的自己省下更多的時間才是最佳選擇。

同一時期，我開始注意到許多開源專案的繁體中文翻譯品質參差不齊，裡頭摻雜了大量中國大陸用語，甚至完全沒有繁體中文版本。這個觀察成為後來投入開源翻譯的起點，也是 **replace.sh** 這個工具誕生的遠因。

自動化的實踐

在簡體中文翻譯良率高的專案中，可以利用 **OpenCC** 直接將簡體初步轉換成繁體中文。不過在利用這個工具一段時間後發現，某些用詞會有嚴重的台陸差異必須替換，例如「**公交**」「**公車**」等。於是我著手寫腳本從根本解決問題，將重複度高的工作自動化。以 **OpenCC** 為基底，**sed** 的用詞轉換字典檔為輔，一個簡易的 **replace.sh Bash** 腳本就誕生。

在腳本的基礎功能完成後，我又在一次次發現其他 **usecase** 後新增更多功能。最後不但能處理檔案名稱轉換，又陸續新增了 **PNG metadata**、覆寫警告、**gitignore** 等功能，讓腳本在不破壞原始功能的情況下更加讓其盡善盡美。這支工具讓我能以穩定的品質大量提交翻譯修正，至今已在簡體中文為基底的開源專案如 **NewAPI** 在短時間內為其新增台灣繁中支援了。

同一套「遇到重複的事就自動化」的工程師思維也延伸到其他地方。我寫了 **yt.py** 來自動下載 **YouTube** 播放清單並處理音樂標籤與封面嵌入，再搭配自架的 **Navidrome** 伺服器統一管理音樂庫。這個過程讓我開始接觸 **Docker** 容器化部署，最終在自己的機器上架設了包含 **AdGuard**、**Immich**、**Syncthing** 在內的十餘項服務。從單一腳本到完整的自架生態系，這段歷程讓我確立了一個模式：發現問題 → 寫工具 → 規模化 → 分享給需要的人。

踩坑與調整

高三上學期，我將大量時間投入英文能力的鞏固與技術專案的維護，相對壓縮了其他科目的學習時間，導致物理科的表現不如預期。這段經歷讓我正視一個事實：專注在自己擅長的領域是優勢，但完全忽略不擅長的領域就變成了盲點。進入高三下學期後，我重新調整了時間分配，在維持技術學習節奏的同時補回被忽略的基礎科目。方向沒有改變，改變的是我對「怎麼走」的判斷。

現在

回頭看這三年，從國中拆開 **iOS**、高一寫第一支團隊專案、高二打造 **1800** 行的翻譯工具到高三維運十多項自架服務，串起了未來當工程師的基底。這些並不屬於一門課程，也沒有人要求我要做什麼，而純粹是樂於吸收知識並且回饋給社會的態度所造就的成果。我希望帶著這個習慣進入大學，在更大的系統、更複雜的問題裡，繼續把同樣的事情，做得更大。